NCollection ERP — Local Dev & Architecture (Start Here)

> **Who this is for**: anyone joining the project. It answers "what am I looking at, where does everything live, how do I run and edit it, and how does the team ship?" Read it once end-to-end; keep the Command cheat-sheet handy.

---

1. The single most important thing: there are TWO "websites"

New developers get confused here, so let's kill the confusion immediately. The repo contains **two completely separate things**:

	The Odoo product	The `demo/` React app
What it is	The **real client product**	A **throwaway visual prototype**
Technology	Odoo 19 (Python + XML + OWL + SCSS)	React + TypeScript + Vite
You run it with	`docker compose up -d` → `localhost:8069`	`cd demo && npm run dev` → `localhost:5173`
Data	A **real PostgreSQL database**	**Fake hardcoded** "dummy" data
Lives in	`custom_addons/`	`demo/`
Wired into Docker?	**Yes** — this is the product	**No** — 100% standalone

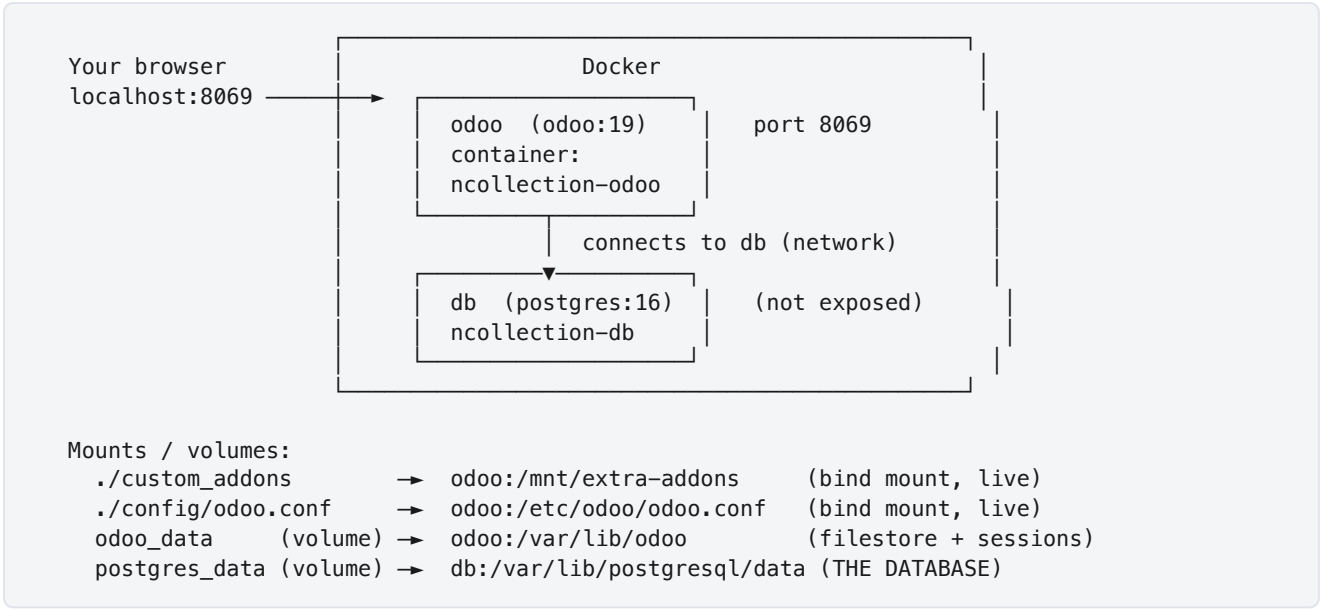
The `demo/` React app is NOT the client's website. It was built to *show stakeholders the intended look* while the real backend is still being built. It is never deployed. Over the Phase-1 tickets its design gets **re-implemented inside Odoo** as OWL components + QWeb templates + SCSS in `custom_addons/`. Treat `demo/` like a coded Figma mockup.

The real product = Odoo + our `custom_addons/`. Everything below is about that, unless it explicitly says "demo".

---

2. Runtime architecture

`docker compose up -d` starts two containers:



- `odoo` **container** runs the official `odoo:19` image (no custom build). It serves the web app on port **8069**.
- `db` **container** runs `postgres:16`. It is **not** exposed to your host — only the `odoo` container can reach it over the Docker network.
- The **database physically lives in the `postgres_data` Docker volume**, not in the repo. That's why you can't "find the database" as a file — Docker manages it.
- `odoo_data` holds Odoo's filestore (uploaded files/attachments) and sessions.
- Your **custom addon code is bind-mounted live** from `./custom_addons` into the container at `/mnt/extra-addons` (Odoo's extra-addons path). Editing a file on your machine changes it inside the container instantly — but Odoo still needs a **module upgrade** to *apply* it (see §6).

---

3. Where everything lives

Thing	Location
The database (data)	`postgres_data` Docker volume (inside the `db` container)
Backend logic & data models	`custom_addons/<module>/models/*.py` (Python)
Frontend (screens)	`custom_addons/<module>/views/*.xml`, `static/src/*.js` (OWL), `static/src/*.scss`
Access rules / security	`custom_addons/<module>/security/`
Menus	`custom_addons/<module>/views/menus.xml`
Odoo dev config	`config/odoo.conf`
Local secrets/creds	`.env` (you create it from `.env.example`; gitignored)
Stack definition	`docker-compose.yml`
The React prototype	`demo/` (separate, dummy data)

Planning docs / tickets	`docs/markdown/` , GitHub Issues

What the four custom modules are today

Module	State	What it is
`ncollection_subscription`	**Real / substantive**	The product core so far: Tenant, Subscription, Plan, Provisioning, and SaaS-admin Dashboard models + views + security
`ncollection_branding`	Small / real	Theme, logo, colors (SCSS + templates)
`ncollection_core`	**Empty skeleton**	Placeholder — filled in by Phase-1 tickets (roles, workspace config, license enforcement)
`ncollection_saas`	**Empty skeleton**	Placeholder — provisioning engine / SaaS admin (Phase 2)

---

4. The Odoo mental model: one monolith, not separate front/back

This is the biggest shift if you come from a React-frontend / API-backend world:

In Odoo there is no separate frontend app and backend API to deploy. Odoo is a **monolith**. A single feature/ticket usually lives inside **one module** and contains *both* layers together:

- **Backend** = Python: `models/*.py` (data + business logic), `controllers/*.py` (web routes, when needed)
- **Frontend** = `views/*.xml` (screens), `static/src/*.js` (OWL components), `static/src/*.scss` (styles)

So when you pick up a task, you typically edit Python **and** XML/OWL/SCSS in the *same* addon folder. The DEV-1 / DEV-2 / DEV-3 split in the plan is about **who owns which tickets**, not separate codebases or repos. The `demo/` React look becomes Odoo OWL/QWeb — same design, different (Odoo-native) technology.

---

5. First-time setup (5 minutes)

```
# 1. (optional) create your local env file — the stack also runs without it
cp .env.example .env

# 2. start the stack
make up            # == docker compose up -d   → Odoo on http://localhost:8069

# 3. create a database and install our modules in one shot
make bootstrap db=ncollection
#   creates the 'ncollection' database and installs
#   ncollection_subscription + ncollection_branding

# 4. open the app
#    http://localhost:8069  → pick the 'ncollection' database → log in
```

Default admin login on a fresh Odoo database is `admin` / `admin`. **Database-manager master password** (to create/drop DBs at `http://localhost:8069/web/database/manager`) is `ncollection_dev` (set in `config/odoo.conf`, dev-only).

> Don't want the one-shot? You can instead open `http://localhost:8069`, use the database manager to create a DB, then install modules from **Apps** (search "NCollection", enable developer mode first — see §6).

---

6. The daily edit loop (no image rebuilds!)

You **do not rebuild the Docker image** to change code. The image (`odoo:19`) is pulled once; your addons are mounted live. The loop is **edit → upgrade → refresh**:

```
# 1. edit files in custom_addons/<module>/ in your editor

# 2. apply the change to the running app:
make upgrade m=ncollection_subscription db=ncollection
#   (upgrades the module against your DB and restarts Odoo)

# 3. refresh the browser
```

What needs what:

You changed…	To see it
Python (`models/*.py`)	`make upgrade m=<module>`
XML views / menus / security	`make upgrade m=<module>`
JS (OWL) / SCSS assets	`make restart` (or turn on developer mode's asset auto-reload)

Only data you typed in the UI	just refresh — it's already in the DB

Enable developer mode (needed for the Apps menu, technical menus, and faster asset reloads): log in → Settings → scroll down → *Activate the developer mode*. Or visit `http://localhost:8069/web?debug=1`.

You rebuild/recreate containers only when you change `docker-compose.yml` or `config/odoo.conf` (`make up` recreates as needed), never for day-to-day code edits.

---

7. How the team ships: GitHub workflow

There are **100 GitHub issues** — one per planned task (e.g. `[P1-T07]`). The loop:

```
1. Pick an issue on GitHub              (it has scope + acceptance criteria)
2. git checkout develop && git pull
3. git checkout -b feature/p1-t07-short-description   (branch OFF develop)
4. Edit custom_addons/, test locally with `make upgrade`
5. git commit, git push -u origin <branch>
6. Open a Pull Request  →  base branch: develop
7. CI runs automatically (4 checks, see below)
8. One teammate reviews → merge into develop
```

- **`develop`** = shared integration branch (everyone merges here first).
- **`main`** = stable. `develop` → `main` when a release is cut.
- Use the **task-prompt template** (`docs/markdown/TASK_PROMPT_TEMPLATE.md`) when you start a ticket — especially if you're driving it with an AI assistant.

CI (runs on every Pull Request)

Defined in `.github/workflows/ci.yml`. Four jobs, all **validation only**:

Job	What it checks
`lint`	`flake8` + `pylint-odoo` (fails only on *new* findings above the baseline)
`architecture-guard`	project rules — Odoo 19 view syntax, two-layer separation, no hardcoded secrets, license-gate-needs-ORM-check (`scripts/ci/architecture_guard.py`)
`test`	spins up Postgres, installs all modules with `--test-enable`, fails on any traceback
`build`	`docker compose up -d` + smoke-tests that `/web/login` responds

Deployment

There is no automated deploy (CD) yet. CI only *validates* PRs. Standing up staging and production servers is upcoming Phase-2/Phase-3 work (`P2-T07` staging, `P2-T08` hardening, `P3-T13` go-live). For now, "running it" means running it locally as above.

---

8. Where the project actually is right now

You're joining at the **foundation**. What exists today is close to **stock Odoo** plus a subscription module and branding. Deliberately **not built yet** (they are the first tickets):

Not there yet	Ticket that adds it
Multi-tenant routing (`db_filter` , subdomain → tenant DB)	**P1-T02**
Production secrets, worker tuning, hardened `odoo.conf`	**P1-T02**
Nginx reverse proxy + TLS	**P1-T03**
The customer OWL dashboard, roles, license enforcement	P1-T07 … P1-T17
Staging / production deploy	P2 / P3

So the "plain / empty" feeling is expected — that scaffolding is exactly what the early tickets build. The `config/odoo.conf` in this repo is a **dev bootstrap**; its production version is owned by P1-T02 (see the commented TODOs inside that file).

---

9. The dev-config files (what each one is)

File	Purpose
`docker-compose.yml`	Defines the two containers, volumes, ports, and mounts. Reads DB creds as `${VAR:-default}` , so it runs with **or without** a `.env` .
`config/odoo.conf`	Local-dev Odoo config (addons path, dev master password, `list_db=True` , threaded mode). Production/multi-tenant settings are commented TODOs → **P1-T02**.
`.env.example`	Template. `cp .env.example .env` to override local DB creds. **`.env` is gitignored — never commit real secrets.**
`Makefile`	Shortcut commands (below). Thin wrappers over `docker compose` .

---

10. Command cheat-sheet

Run `make help` any time. Common ones:

```
make up                 # start the stack (Odoo on :8069)
make down               # stop & remove containers (keeps your data)
make restart            # restart Odoo (apply JS/SCSS or config changes)
make logs               # follow Odoo logs (Ctrl-C to exit)
make ps                 # container status

make bootstrap db=ncollection      # first run: create DB + install our modules
make install m=<module> db=<db>    # install a module
make upgrade m=<module> db=<db>    # apply code edits to a module (+ restart)
make createdb db=<db>              # create an empty Odoo DB
make dropdb db=<db>                # delete a DB (destructive)

make shell              # bash inside the Odoo container
make psql db=<db>       # psql prompt on a database
make odoo-shell db=<db> # Odoo Python shell (env, models, ORM)

make demo               # run the SEPARATE React prototype on :5173
```

- `db=` defaults to `ncollection`; pass `db=<name>` to target another database.
- `m=` is the module technical name, e.g. `ncollection_subscription`.

Equivalent raw commands (if you prefer not to use `make`) are just the `docker compose …` lines each target wraps — see the `Makefile`.

---

11. Quick FAQ

Where's the database? In the `postgres_data` Docker volume, inside the `db` container. Not a file in the repo. Created when you make your first Odoo DB.

Why do I see plain Odoo, not our product? Because no modules are installed on a fresh DB. Run `make bootstrap`, or install them from **Apps**.

Do I rebuild the image when I edit code? No. Addons are mounted live; run `make upgrade m=<module>` and refresh.

Is `demo/` the client website? No — it's a standalone React prototype with fake data. The real product is Odoo + `custom_addons/`.

How do I deploy? You don't yet — there's no CD. Local only until Phase 2/3.